



Software Design Description

Pi Panel hardware control and temperature
monitoring application

Document ID: PI-PANEL-SDD

Baseline: 1.0

Status: Approved design snapshot

Snapshot date: 14 September 2026

Platform: Raspberry Pi 5 / Explorer HAT Pro / Qt 6 / C++20

Document Control

Purpose	Describe the implemented software architecture, interfaces, behavior, constraints, and verification strategy.
Scope	The <code>pi-panel</code> repository at baseline 1.0.
Audience	Developers, maintainers, integrators, and reviewers.
Source of truth	Checked-in source code and tests. This PDF is a controlled snapshot.
Refresh policy	Regenerate this PDF only when the project owner explicitly requests a documentation refresh.

Table of Contents

01	Introduction	3
02	System context and constraints	4
03	Architecture	5
04	Component design	6
05	Interfaces and contracts	7
06	Dynamic behavior	8
07	State and configuration	9
08	Reliability and quality attributes	10
09	Build and deployment design	11
10	Verification and evolution	12

1. Introduction

1.1 Purpose

This Software Design Description (SDD) records how Pi Panel is structured and how its implemented parts collaborate. It is intended to support maintenance, review, testing, and future feature design.

1.2 Product scope

Pi Panel is a single-process graphical application running on a Raspberry Pi 5. It provides:

- Control of an Explorer HAT Pro onboard LED.
- Debounced input from a physical button connected to Explorer HAT Input 1.
- Voltage acquisition from Explorer HAT Analog 1.
- Conversion of thermistor voltage into degrees Fahrenheit.
- Display of LED state, voltage, temperature, and recoverable measurement errors.

1.3 Out of scope

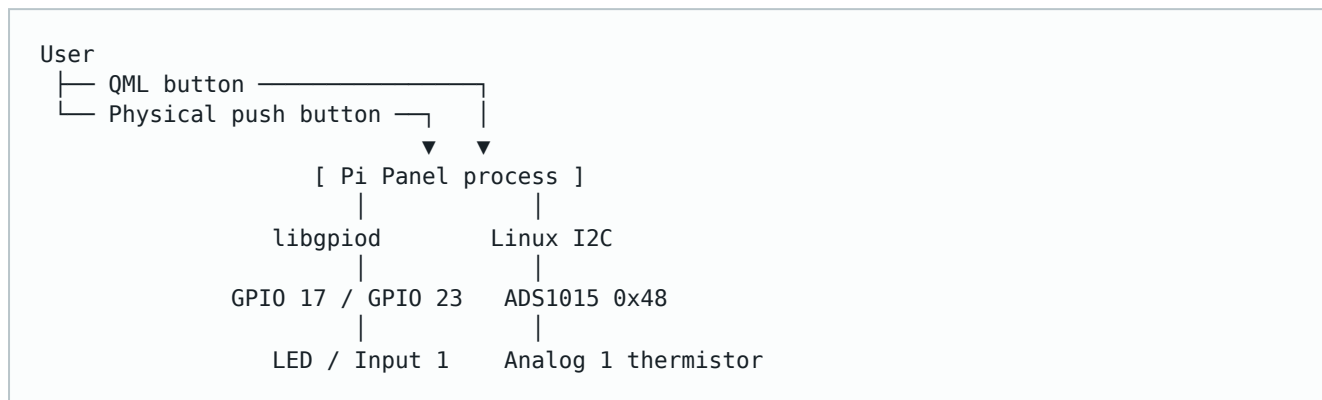
- Network services, remote APIs, user accounts, and persistent storage.
- Automatic service startup at boot.
- Closed-loop temperature control or safety interlocks.
- Automatic thermistor calibration or sensor identification.

1.4 Design principles

- Keep hardware access behind narrow C++ abstractions.
- Keep QML declarative and free of hardware details.
- Use timer-driven work on the Qt event thread for the current small workload.
- Convert hardware failures into explicit UI state or controlled termination.
- Make pure logic independently testable without Raspberry Pi hardware.

2. System Context and Constraints

2.1 External context



2.2 Hardware constraints

Resource	Constraint
LED	Explorer HAT LED 2 maps to BCM GPIO 17.
Button	Explorer HAT Input 1 maps to BCM GPIO 23 and is active-high.
Input bias	The HAT buffer prevents Pi internal bias from controlling the external input; an external pull-down is required.
ADC	ADS1015 at I2C address 0x48 ; Analog 1 maps to ADC channel 3.
Thermistor	Default model assumes 10 kΩ NTC, $\beta=3950$ K, 25°C nominal, and a 3.3 V divider.

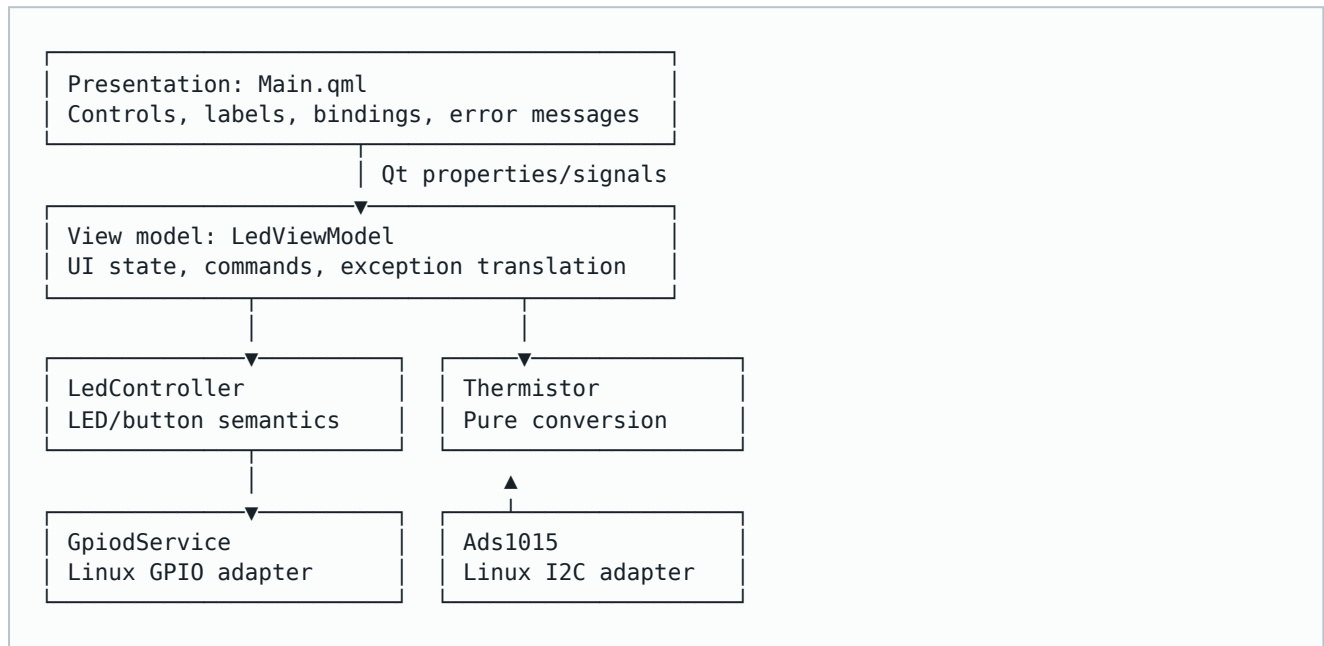
2.3 Software constraints

- Linux with `libgpiodcxx` and the Linux I2C device API.
- Qt 6 Core, Gui, Qml, and Quick.
- C++20 with compiler warnings treated as errors.
- The application user requires access to GPIO and the selected `/dev/i2c-*` device.
- Only one process may own the requested GPIO lines.

3. Architecture

3.1 Architectural style

The application uses a small layered, event-driven design. QML is the presentation layer, `LedViewModel` is the presentation boundary, controllers contain device-oriented behavior, and services perform operating-system I/O.



3.2 Dependency direction

- QML depends on the view-model contract, not controller types.
- The view model depends on controller and conversion objects by reference; `main.cpp` owns their lifetime.
- `LedController` depends on `IGpioService`, allowing a mock implementation in tests.
- `Thermistor` has no Qt or hardware dependency.
- `Ads1015` and `GpiodService` are Linux-specific boundary adapters.

3.3 Concurrency model

The current application is single-threaded at the application level. Both timers execute callbacks on the Qt event thread. ADC conversion includes short sleeps while waiting for hardware, bounded by a 100 ms timeout.

4. Component Design

Component	Responsibilities	Owned state/resources
<code>main</code>	Composition root, signal registration, timer scheduling, QML engine setup, shutdown.	Controllers, view model, timers, Qt application.
<code>GpiodService</code>	Request LED and button lines; read/write logical values.	<code>gpiod::line_request</code> .
<code>LedController</code>	Track LED state; map toggle/off commands to GPIO; interpret active-high button.	Pin numbers, LED state, shared GPIO service.
<code>ButtonDebouncer</code>	Accept a press only after stable input for the configured interval.	Last raw reading, stable state, last transition time.
<code>Ads1015</code>	Open I2C lazily; configure conversion; wait; decode voltage.	I2C path, address, file descriptor.
<code>Thermistor</code>	Validate configuration and convert voltage to Fahrenheit.	Immutable conversion configuration.
<code>LedViewModel</code>	Expose QML properties, invoke controllers, maintain availability/error state.	References and current UI-facing values.
<code>Main.qml</code>	Render state and forward the toggle command.	Visual object state only.

4.1 Lifetime and ownership

`main.cpp` constructs objects in dependency order: `LedController`, `Ads1015`, `Thermistor`, then `LedViewModel`. They remain on the stack until the Qt event loop exits. The view model stores references, so it cannot outlive those dependencies.

4.2 Resource acquisition

GPIO lines are requested during `GpiodService` construction. The ADC file descriptor is opened lazily on the first read and closed by `Ads1015` destruction. Copying `Ads1015` is disabled to prevent duplicate descriptor ownership.

5. Interfaces and Contracts

5.1 Internal C++ interfaces

Operation	Contract
<code>IGpioService::setLineValue</code>	Write a logical value to a requested output line; may throw on OS/hardware failure.
<code>IGpioService::getLineValue</code>	Read a requested input line; may throw on OS/hardware failure.
<code>LedController::toggle</code>	Write the inverse output and update stored state only after the write succeeds.
<code>ButtonDebouncer::update</code>	Return true only when a stable transition into pressed state reaches the debounce interval.
<code>Ads1015::readVoltage</code>	Accept channels 0-3; return non-negative volts; throw on invalid channel, I/O failure, or timeout.
<code>Thermistor::temperatureFahrenheit</code>	Require voltage strictly between zero and supply; return beta-model temperature.

5.2 QML-facing contract

Property/command	Meaning
<code>ledOn / toggle()</code>	Current controller state and user command.
<code>analogVoltage</code>	Most recent successful Analog 1 sample.
<code>analogAvailable</code> , <code>analogError</code>	ADC status and diagnostic text.
<code>temperatureFahrenheit</code>	Most recent successful conversion.
<code>temperatureAvailable</code> , <code>temperatureError</code>	Conversion status and diagnostic text.

5.3 External interfaces

- GPIO character device through `libgpiodcxx`.
- Linux I2C device through `open`, `ioctl(I2C_SLAVE)`, `read`, and `write`.
- Qt platform plugin through the selected desktop environment.
- Process environment variables for thermistor and I2C configuration.

6. Dynamic Behavior

6.1 Startup sequence

```
QGuiApplication
→ register POSIX signal handlers
→ construct GPIO/LED, ADC, thermistor, and view model
→ seed ButtonDebouncer from current Input 1 state
→ connect 5 ms and 200 ms timers
→ load Main.qml with ledViewModel initial property
→ take initial analog sample
→ start timers and enter Qt event loop
```

6.2 Physical button sequence



Raw changes restart the debounce interval. The stable press transition returns true once. Stable release is recorded but returns false, preventing a second toggle.

6.3 Analog sample sequence

```
QTimer timeout
→ LedViewModel::sampleAnalogInput()
→ Ads1015::readVoltage(channel 3)
→ update voltage and analog status
→ Thermistor::temperatureFahrenheit(voltage)
→ update temperature and conversion status
→ Qt signals refresh QML bindings
```

6.4 Shutdown sequence

SIGINT, SIGTERM, or SIGHUP sets a signal-safe flag. The next 5 ms timer callback calls `application.quit()`. After the event loop returns, timers stop and the LED is explicitly turned off. Object destructors then release I2C and GPIO resources.

7. State and Configuration

7.1 Runtime state

Owner	State	Update rule
LedController	m_ledOn	Changed after a successful GPIO write; cleared by <code>turnOff()</code> .
ButtonDebouncer	Raw reading, accepted state, change time	Updated on every poll according to elapsed steady-clock time.
LedViewModel	Voltage, temperature, availability flags, error strings	Updated after each sample attempt.
Ads1015	File descriptor	Opened once on first use; retained until destruction.

7.2 Configuration inputs

Variable	Default	Validation
PI_PANEL_I2C_DEVICE	/dev/i2c-1	Validated when opened.
PI_PANEL_DIVIDER_VOLTS	3.3	Must parse as positive.
PI_PANEL_FIXED_RESISTOR_OHMS	10000	Must parse as positive.
PI_PANEL_THERMISTOR_NOMINAL_OHMS	10000	Must parse as positive.
PI_PANEL_THERMISTOR_NOMINAL_C	25	Must parse as a number.
PI_PANEL_THERMISTOR_BETA	3950	Must parse as positive.
PI_PANEL_THERMISTOR_TO_GROUND	1	Exactly 0 selects the opposite orientation.

7.3 Persistence

The application has no persistent runtime data. Configuration is supplied at process launch, and measurements exist only in memory. A restart resets LED state, availability state, and sampled values.

8. Reliability and Quality Attributes

8.1 Error policy

Failure	Response	Impact
LED command failure	Emit <code>operationFailed</code> ; do not emit <code>state-changed</code> .	Application remains active.
ADC open/read/write/timeout	Set analog unavailable and expose message.	LED remains usable; conversion is skipped.
Invalid thermistor voltage	Set temperature unavailable and expose message.	Raw voltage remains visible.
GPIO polling failure	Log and exit with status 1.	Application terminates and performs normal cleanup.
Invalid startup configuration	Top-level catch logs and returns 1.	Application does not start.

8.2 Timing and responsiveness

- Button poll: 5 ms; accepted press latency is normally 30–35 ms plus event-loop scheduling.
- Analog update: 200 ms nominal period.
- ADC wait: bounded at 100 ms, but runs on the UI thread and can temporarily delay rendering.

8.3 Safety and security considerations

- The software is monitoring/control software, not a certified safety system.
- Hardware voltage limits and Explorer HAT ratings remain external constraints.
- Device access relies on Linux user/group permissions; the application does not require embedded credentials.
- Environment input is parsed at startup; malformed numeric values fail closed by preventing startup.

9. Build and Deployment Design

9.1 Build products

Target	Contents
pi_panel_core	ADC, debounce, GPIO, LED, and thermistor C++ implementation.
pi_panel	Qt executable, view model, and embedded QML module.
pi_panel_tests	GoogleTest executable linked to the core library.

9.2 Build dependencies

- CMake 3.20 or newer and a C++20 compiler.
- Qt 6 Core, Gui, Qml, and Quick development packages.
- pkg-config and libgpiodcxx.
- GoogleTest 1.14 fetched by CMake when configuring tests.

9.3 Runtime environment

- /dev/gpiochip0 with access to lines 17 and 23.
- /dev/i2c-1 or configured replacement, with ADC address 0x48.
- A Qt-supported display. The checked-in profiles use the Pi Xwayland display or PuTTY-forwarded X11.

9.4 Deployment model

The executable is launched interactively from VS Code or a terminal. No systemd unit, installer, update agent, or automatic boot startup is part of the current design.

10. Verification and Evolution

10.1 Automated verification

Area	Verified behavior
LED controller	Initial state, toggle transitions, pin values, explicit off, destructor cleanup, button polarity, custom pins.
Debouncer	Stable press, ignored early press, bounce reset, release behavior, repeated cycles.
Thermistor	Nominal midpoint, expected direction for both orientations, range rejection, observed 1.70 V room reading.

10.2 Current test gaps

- No automated I2C transaction tests for `Ads1015`.
- No unit tests for `LedViewModel` signal and error-state transitions.
- No automated QML interaction or screenshot tests.
- No integration test covering real GPIO and ADC hardware together.

10.3 Extension guidance

- Add new hardware behind an interface when practical, preserving testability.
- Keep physical units explicit in names and contracts.
- Move blocking I/O off the UI thread if sampling becomes slower or more complex.
- Add persistent configuration only when runtime environment variables become insufficient.
- Update Markdown continuously as appropriate, but regenerate this PDF only on explicit request.

Baseline rule: This document describes the implementation as of 14 September 2026. Later code changes do not implicitly revise this SDD.